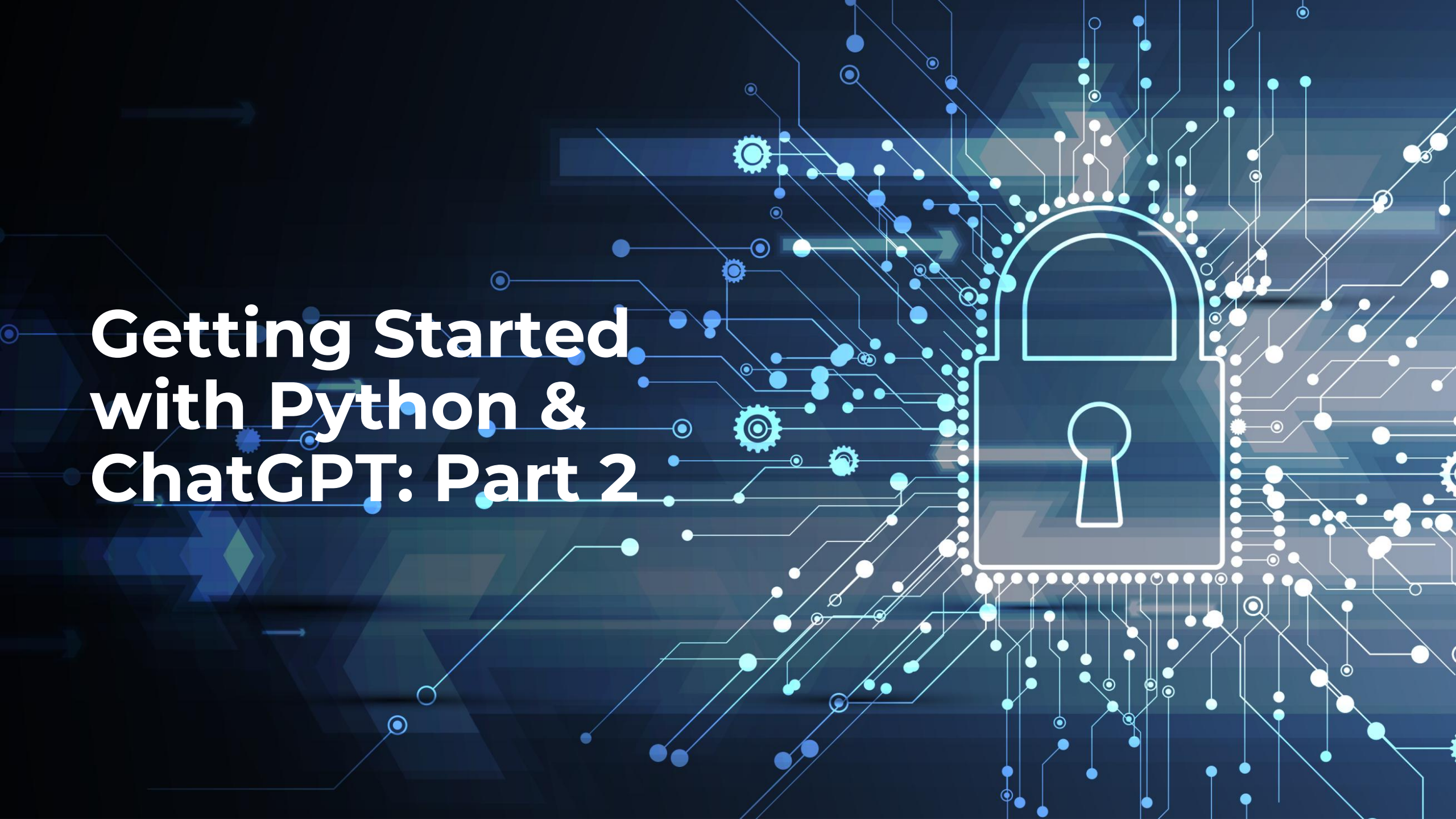
IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Getting Started with Python & ChatGPT: Part 2



Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How AI evolved from rule-based systems to modern intelligent models
- What Agentic AI is and how autonomous systems differ from traditional AI
- The key limitations and advantages of today’s AI systems
- Real-world applications that demonstrate how Agentic AI adds value
- The core components that enable Agentic AI to plan and act autonomously
- How to get started with ChatGPT and Python for basic AI workflows
- The fundamentals of writing simple Python scripts for AI tasks

Topics Covered

- Prompting, Debugging & Reflection with ChatGPT
- Prompt Engineering
- Debugging with AI Support
- Group Exercise
- Pitfalls of ChatGPT
- Activity

Prompting, Debugging & Reflection with ChatGPT

Activity 1: Prompt-Based Real-World Task

Requirement

EMI Calculator:

Develop a prompt to create a functional loan calculator with clear explanations.

Activity 1: Prompt-Based Real-World Task

Prompt 1

You are an expert Python developer and instructor.

Write a complete Python programme for a Loan Calculator that allows the user to input loan amount, annual interest rate and loan tenure (in years or months).

The programme should calculate and display **Monthly Payment (EMI), Total Payment (Principal + Interest), Total Interest Paid**.

Activity 1: Prompt-Based Real-World Task

Prompt 1

You are an expert Python developer and instructor. Write a complete Python program for a Loan Calculator that allows the user to input loan amount, annual interest rate, and loan tenure (in years or months). The program should:

Calculate and display:
Monthly Payment (EMI)
Total Payment (Principal + Interest)
Total Interest Paid

Here's a **complete Python program** for a Loan Calculator that fulfills your requirements.

Prompt-Based Real-World Task

Complete Python Programme for Loan Calculator

```python
# Loan Calculator Program

def loan_calculator():
    print("==== Loan Calculator ====")

    # User inputs
    loan_amount = float(input("Enter loan amount: "))
    annual_interest_rate = float(input("Enter annual interest rate (in %): "))

    tenure_type = input(
        "Is the tenure in years or months? (Enter 'Y' for years or 'M' for months): "
    ).strip().upper()

    if tenure_type == 'Y':
        tenure_months = int(input("Enter loan tenure in years: ")) * 12
    elif tenure_type == 'M':
        tenure_months = int(input("Enter loan tenure in months: "))
    else:
        print("Invalid input! Please restart and enter 'Y' or 'M'.")
        return

    # Convert annual interest rate to monthly
    monthly_interest_rate = annual_interest_rate / 12 / 100

    # EMI Calculation using formula
    if monthly_interest_rate > 0:
        emi = (
            loan_amount * monthly_interest_rate *
            (1 + monthly_interest_rate) ** tenure_months
        ) / ((1 + monthly_interest_rate) ** tenure_months - 1)
    else:
        # If interest rate is 0
        emi = loan_amount / tenure_months
```

Prompt-Based Real-World Task

Prompt 2

Explain every code statement in simpler term

Got it 👍

I'll go through the **Loan Calculator program line by line** and explain everything in **simpler terms**.

🔎 Explanation of the Loan Calculator Code

python Copy Edit

```python
# Loan Calculator Program
```

👉 This is just a comment (not executed). It tells us that the program is a Loan Calculator.

python Copy Edit

```python
def loan_calculator():
```

👉 Defines a **function** called `loan_calculator`.

Activity 2: Prompt-Based Real-World Task

Requirement

To-Do list generator

Create a prompt to create a simple to-do list app in python.

Activity 2: Prompt-Based Real-World Task

Prompt

✓ You are an expert Python tutor. Write a complete Python programme for a Simple To-Do List App that works in the terminal.

✓ The app should

- Allow the user to add new tasks.
- Allow the user to view all tasks with numbering.
- Allow the user to mark a task as completed (and show it as completed in the list).
- Allow the user to delete a task.
- Keep running in a loop until the user chooses to exit

Activity 2: Prompt-Based Real-World Task

Prompt

You are an expert Python tutor. Write a complete Python program for a Simple To-Do List App that works in the terminal. The app should:
Allow the user to add new tasks.
Allow the user to view all tasks with numbering.
Allow the user to mark a task as completed (and show it as completed in the list).
Allow the user to delete a task.
Keep running in a loop until the user chooses to exit.

Great! Let's build a **Simple To-Do List App in Python (terminal version)** step by step.
I'll write clean code with comments so you can follow along easily.

📌 Complete Python Program: To-Do List App

python Copy Edit

```python
# Simple To-Do List App (Terminal Version)

def show_menu():
    """Display the menu options."""
```

Activity 2: Prompt-Based Real-World Task

Prompt

```
==== To-Do List Menu ====
1. View Tasks
2. Add Task
3. Mark Task as Completed
4. Delete Task
5. Exit
Choose an option (1-5):
```

Activity 2: Prompt-Based Real-World Task

Refining the Prompt

Use a menu-driven interface (e.g., numbers for choices: 1. Add Task, 2. View Tasks, etc.).

Store tasks in a Python list (no database required).

Add clear inline comments explaining each part of the code so beginners can follow along.

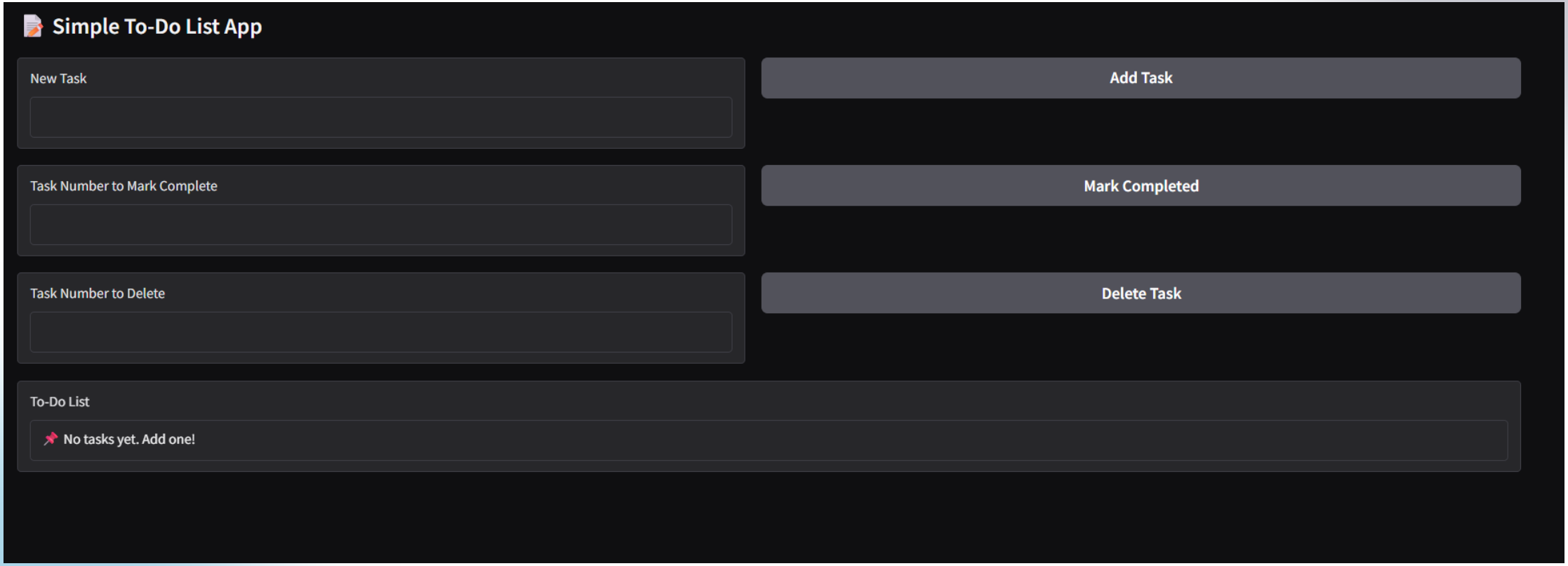
Print messages in a user-friendly format in Gradio App.

Activity 2: Prompt-Based Real-World Task

To-Do- List App in Python (Terminal Version)

```python
# ================================
# Simple To-Do List (Gradio App)
# ================================

import gradio as gr

# Store tasks in memory (list of dicts)
tasks = []


def view_tasks():
    """Return formatted string of all tasks."""
    if not tasks:
        return "✓ No tasks yet. Add one!"

    result = "==== Your Tasks ====\n"
    for idx, task in enumerate(tasks, start=1):
        status = "✅ Completed" if task["completed"] else "❌ Not Completed"
        result += f"{idx}. {task['title']}  [{status}]\n"
    return result


def add_task(task_text):
    """Add a new task."""
    if task_text.strip():
        tasks.append({"title": task_text.strip(), "completed": False})
        return f"✅ Task '{task_text.strip()}' added successfully!\n\n" + view_tasks()
    return "⚠ Task cannot be empty."


def complete_task(task_number):
    """Mark a task as completed."""
    try:
```

Activity 2: Prompt-Based Real-World Task



Prompt Engineering

Prompt Engineering

Why Iteration Matters?

Prompts = Programming for AI

Refine instructions through iteration to achieve accurate AI output

Treat prompting as programming, where clarity and specificity affect results

Optimise prompts like code, using debugging and careful refinement

Techniques for Better Prompts

Specify format, style, length or other parameters to narrow the response scope.

Add constraints

Example: "Return as valid Python code with type hints" or "Limit explanation to 100 words."

Ask for examples that verify the solution works.

Request test cases

Example: "Include 3 edge-case test cases" or "Show sample invoices with missing fields"

Request that ChatGPT explain its approach.

Ask for step-by-step Reasoning

Example: "Walk through your solution" or "Explain each code part"

These techniques improve output and reveals ChatGPT's strengths and limits.

Example – Prompt Iteration

Initial prompt

- **Request:** Write an EMI calculator in Python
- **Outcome:** Basic function, minimal error handling, no explanation

First refinement

- **Request:** Add comments explaining the formula and input validation
- **Outcome:** Improved code with error handling, basic output format

Final refinement

- **Request:** Include input validation, error messages, optional amortisation schedule, formatted output and unit tests
- **Outcome:** Complete solution with validation, visualisation and testing

Instructions

Refine your prompt

Iterate 2–3 times to improve your prompt based on gaps identified in Activity 2.

Test each version

Submit each prompt to ChatGPT and save the responses for comparison.

Analyse improvements

Compare outputs across iterations to assess quality changes and identify effective techniques

Group discussion

Share findings with the group and discuss patterns in successful prompts.

Debugging with AI Support

Activity 3 : Debugging with AI Support

Benefits of AI-Assisted Debugging

Translates cryptic errors into plain English

Suggests multiple fixes with explanations

Identifies logical errors without explicit warnings

Reduces debugging time for syntax and common logical issues

Always validate ChatGPT's fixes, as they may misinterpret context or be incomplete.

Broken Script Example

Code with Errors

```
def add_numbers(a, b) return a +
bprint(add_numbers(5))
```

- This simple function contains two common errors:
  - Missing colon in function definition
  - Incorrect number of arguments in function call

Broken Script Example

Error Message

```
File "example.py", line 1 def add_numbers(a, b)
^SyntaxError: expected ':'# If the first error is
fixed:TypeError: add_numbers() missing 1 required
positional argument: 'b'
```

ChatGPT efficiently diagnoses and explains common errors that are simple for experienced developers.

Debugging with ChatGPT

Example Debugging Prompt

This Python code has errors. Please identify them, explain why they're occurring and provide a fixed version:

```
def add_numbers(a, b) return a +
bprint(add_numbers(5))
```

Debugging with ChatGPT

ChatGPT's Response

- **Issues identified**
  - Missing colon after function parameters
  - Function call missing the second required argument
- **Fixed code**

```
def add_numbers(a, b): return a +
bprint(add_numbers(5, 3))
```

Debugging Scenario: Logical Error in a Loop

Example of Debugging Prompt

This Python code has errors. Please identify them, explain why they're occurring and provide a fixed version:

```python
# Program to calculate the average of numbers

def calculate_average(numbers):

total = 0

for i in range(1, len(numbers)): # Logical bug
here

total += numbers[i]

return total / len(numbers)

data = [10, 20, 30, 40, 50]

print("Average:", calculate_average(data))
```

Debugging Scenario: Logical Error in a Loop

Prompt

- **Issues identified:**
  - This Python code gives the wrong average. Can you find the bug, explain it and fix the function?
- **Likely response**

```
Identifies the off-by-one error in the loop (range(1,
len(numbers)) should be range(len(numbers))).

Suggests alternative fixes:Start loop at 0Or simply
use Python's built-in sum() function.
```

Debugging Scenario: Logical Error in a Loop

Corrected Code

```
def calculate_average(numbers):

total = 0

for i in range(len(numbers)): # Fixed loop

total += numbers[i]

return total / len(numbers)

.

# Or a more Pythonic fix:

def calculate_average(numbers):

return sum(numbers) / len(numbers)

.

print("Average:", calculate_average([10, 20, 30, 40,
50])) # Output: 30.0
```

Group Exercise

Group Exercise

Practice ChatGPT Using Real Debugging Scenarios

Identify issues

Analyse a provided broken code snippet using ChatGPT to spot multiple errors.

Implement fixes

Apply ChatGPT's suggested corrections in Jupyter Notebook or your preferred environment.

Discuss limitations

Reflect with your group on when ChatGPT was helpful and when it fell short.

Test & verify

Run the fixed code to ensure it works and check for any missed errors.

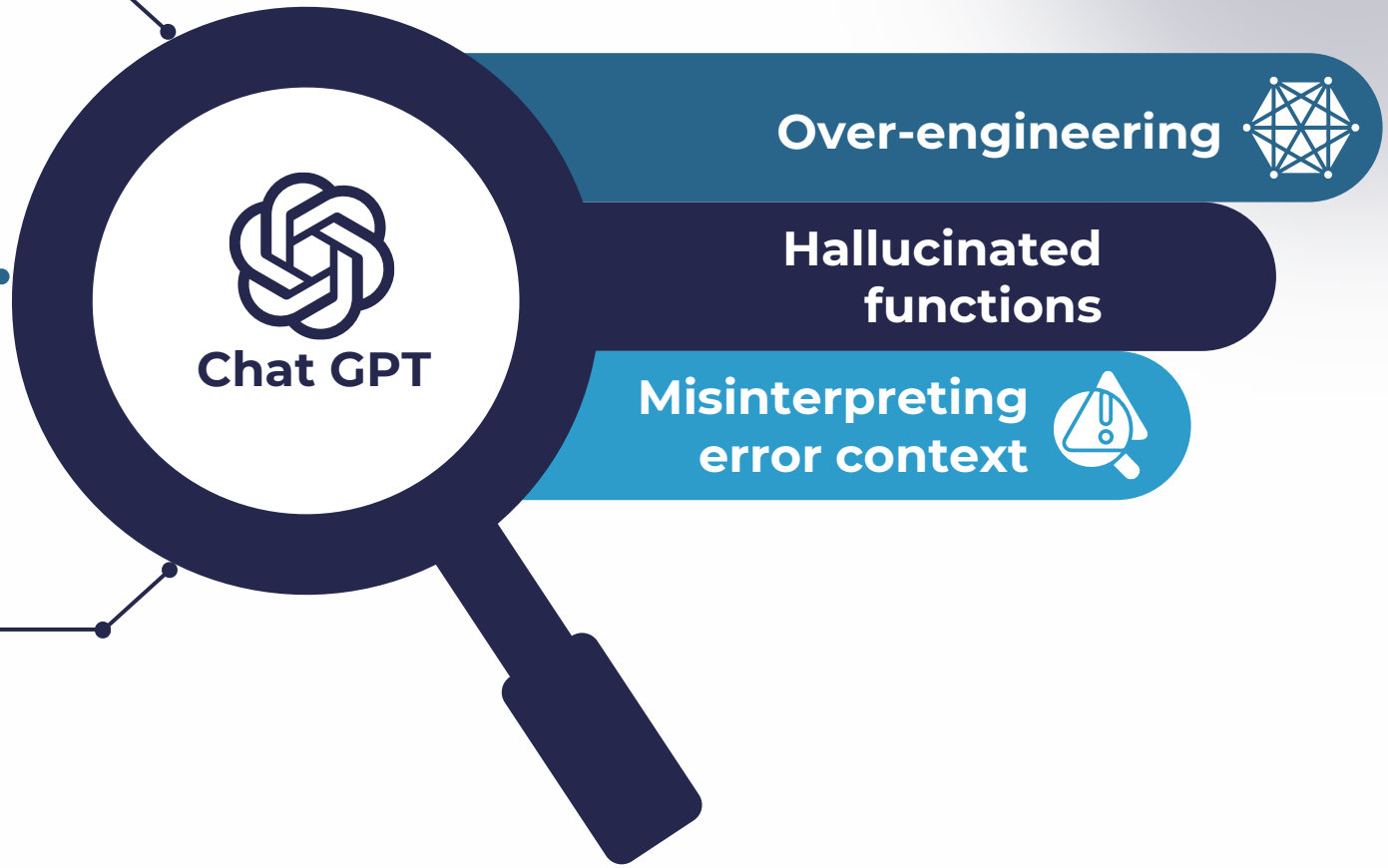
Pitfalls of ChatGPT

Pitfalls of ChatGPT

ChatGPT may overcomplicate simple problems, especially with vague prompts

ChatGPT may cite plausible but non-existent libraries or functions

Without full context, ChatGPT may misdiagnose errors and suggest superficial fixes



Understanding these limitations helps you develop more effective strategies for working with AI tools.

How to Evaluate Responses

Verification Checklist

- Code runs successfully without errors
- Solution handles edge cases appropriately
- Output matches requirements exactly
- Explanation is clear and matches the code
- Uses real, documented libraries and functions
- Performance is acceptable for the use case

Always validate ChatGPT's confident answers with docs, data and experts.

Activity

Activity

Steps for Using Chat GPT for Job Automation

- Identify 3 role-specific AI applications.
- Note for each: time/effort savings, risks, verification method.
- Choose one application to implement within a week.

This reflection links the workshop to your workflow, enhancing practical application.

Key Takeaways

Start with clear prompts

Provide specific, detailed instructions, including formats, constraints and expectations.

Reflect on application

Assess effectiveness regularly and adapt based on results.

Iterate deliberately

Refine prompts to address gaps in previous responses.

Debug with guidance

Use ChatGPT for error explanations and fixes, but always test and validate yourself.

Use ChatGPT alongside your expertise for best results.

Q & A

Thank you